Tema 1: Introducció als Sistemes Operatius

1. Definició i Rol del Sistema Operatiu

El **Sistema Operatiu (SO)** és un software que controla els recursos disponibles del sistema hardware i fa d'intermediari entre les aplicacions i el hardware.

- **Visió interna**: Defineix estructures de dades per gestionar el hardware i algorismes de decisió per determinar com utilitzar-lo.
- **Visió externa**: Ofereix un conjunt de funcions per accedir a la seva funcionalitat (o serveis) de gestió de recursos.

---

2. Components del Sistema i Entorn de Treball

Habitualment coneixem com a **"sistema"** la suma del **nucli (*kernel*)** i les **aplicacions d'usuari**.

- **Components principals**:
  - **Kernel**: Part central del SO que s'executa en mode privilegiat i gestiona directament el hardware mitjançant instruccions privilegiades.
  - **Entorn de treball interactiu**: Permet a l'usuari comunicar-se amb el SO. Pot ser un intèrpret de comandes (*shell*) o un entorn gràfic (GUI).
  - **Entorn de desenvolupament i aplicacions**: Editors de text (`gedit`), compiladors (`gcc`), navegadors web o bases de dades que utilitzen les crides a sistema per demanar serveis al kernel.

Cicle de desenvolupament a la Shell

A la línia de comandes (*shell*), el procés tipus per crear i executar un programa en C consta de tres passos consecutius:

```
# 1. Edició del codi font en C
gedit programa.c

# 2. Compilació i enllaçat (linking) amb la llibreria de sistema
gcc -o programa programa.c

# 3. Execució de l'executable a través de la shell
./programa
```

---

3. Objectius del SO i de les Aplicacions

Durant el funcionament del sistema, el SO persegueix tres objectius bàsics, els quals **també han de complir els programes desenvolupats per l'usuari**:

1. **Usabilitat**:
   - *Sistema*: Abstraeix els usuaris de les diferències complexes entre diferents arquitectures.
   - *Programes d'usuari*: Si l'usuari no executa el programa correctament o una crida a sistema falla, el programa ha de donar un **missatge d'error clar i útil**.
2. **Seguretat**:
   - *Sistema*: Protegeix el hardware d'accessos incorrectes i manté la protecció entre diferents usuaris.
   - *Programes d'usuari*: El programa no ha de fallar si l'usuari l'utilitza incorrectament, ha de **controlar totes les fonts d'error** i de cap manera pot fer fallar el sistema.
3. **Eficiència**:
   - *Sistema*: Garanteix un ús eficient dels recursos del sistema. Quan múltiples usuaris hi accedeixen simultàniament, els ofereix la sensació d'un accés en "exclusiva".
   - *Programes d'usuari*: Han d'aprofitar la millor manera els recursos de la màquina en un entorn compartit.

---

4. Cicle de Vida del Sistema

El cicle de vida operatiu del SO es divideix en tres fases principals:

```
[ Arrencada (Boot / Startup) ]  --->  [ Ús Actiu ]  --->  [ Finalització (Shutdown) ]
```

1. **Arrencada (*Boot / Startup*)**:
   - El hardware carrega el SO (o una part d'ell) a la memòria RAM.
   - S'inicialitzen les estructures de dades necessàries (hardware i software) per controlar la màquina.
   - Les **interrupcions hardware** són capturades pel SO.
   - Es posa en marxa el programa d'accés al sistema (*login*, *shell* o entorn gràfic).
2. **Ús**:
   - Funciona com a entorn d'execució de programes existents i com a entorn de desenvolupament per a generar-ne de nous.
3. **Finalització (*Shutdown*)**:
   - Es finalitzen de forma controlada tots els processos actius i s'apaguen o sincronitzen els dispositius d'emmagatzematge.

---

5. Mecanismes d'Accés al Kernel i Modes d'Execució

Modes d'Execució del Hardware

Per garantir la seguretat del sistema, el hardware disposa de com a mínim dos modes d'execució:

- **Mode no-privilegiat (*User Mode*)**: Mode on s'executen les aplicacions d'usuari. No té accés a instruccions privilegiades ni a determinades regions de memòria protegides.
- **Mode privilegiat (*Kernel Mode*)**: Mode on s'executa el nucli del SO. Té accés total a les instruccions de llenguatge màquina privilegiades i a tota la memòria.

Quan s'executa codi del Kernel?

El kernel és un codi **guiat per esdeveniments**. S'executa únicament quan es produeix un d'aquests tres esdeveniments:

Esdeveniment	Tipus	Origen / Causa	Exemples
Interrupció Hardware	Asíncrona	Generada pel hardware extern (entre 2 instruccions màquina)	Teclat, disc, DMA, **interrupció de rellotge**
Excepció Software	Síncrona	Provocada per l'execució d'una instrucció invàlida o errònia	Divisió per zero, fallada de pàgina (*page fault*)
Crida a Sistema	Síncrona	Provocada explícitament per una instrucció per demanar un servei al SO	Llegir o escriure en un fitxer/pantalla (`write`, `read`)

> **Interrupció de Rellotge**: Per evitar que una aplicació d'usuari pugui acaparar la CPU i el SO perdi el control, el SO programa periòdicament la interrupció de rellotge (ex: cada 10 ms) per recuperar el control i executar la planificació del sistema.

---

6. Crides a Sistema i Llibreries de Sistema

Generació d'Executables i Llibreries

El procés de creació d'un executable passa per la compilació del codi font (generant codi objecte) i la fase d'enllaçat (*linking*), on es resolen els símbols enllaçant el programa amb les **llibreries de sistema**.

- **Llibreria de sistema (ex: `libc`)**: S'executa en **mode usuari**. Serveix per aïllar el programador dels detalls de l'arquitectura i facilitar la invocació de les crides a sistema.

- Per exemple, quan un programador crida a `printf()` o `write()`, la llibreria s'encarrega de:
  1. Carregar els paràmetres als registres/pila segons el conveni de l'arquitectura.
  2. Executar la instrucció de salt al kernel (`TRAP`, `sysenter` o `int`).
  3. Recollir el resultat del kernel i homogeneïtzar-lo (en Linux, les crides retornen `-1` en cas d'error).

Taula de Crides a Sistema i Portabilitat

Per aconseguir portabilitat entre diferents versions del SO, les crides a sistema no s'identifiquen amb una adreça de memòria fixa, sinó amb un **identificador numèric** que s'utilitza com a índex en la **taula de crides a sistema** del nucli.

Exemple de codi real (Tema 1): Gestió d'errors en crides a sistema (`write`)

Aquest exemple en C mostra l'ús directe de la crida a sistema d'escriptura `write()` (oferta per la `libc`) aplicant els principis d'usabilitat i seguretat (control d'errors):

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char buf[80];

    sprintf(buf, "Hola! Això és una sortida mitjançant la crida a sistema
write.\n");
    if (write(1, buf, strlen(buf)) == -1){
         perror("Error en la crida a sistema write");
         exit(1);
    }

    return 0;
}
```